

GlobalAI Module 3 Project Report

Monitoring, Auto-Scaling, and Cost Optimization for LLM Platform

Course Hands-on Practice: Scale to Million Users. Target: 99.95% uptime and 40-60% cost reduction while handling 10x traffic spikes.

Submitted artifacts (from starter files): Prometheus config, alert rules, Grafana dashboard JSON, Kubernetes HPA, JMeter load test plan, cost optimizer module, and traffic/prompt datasets.

Scenario snapshot: 500 enterprise clients, ~50M daily requests, latency breaches (30s vs 2s SLA), \$3M monthly cost with low utilization, and revenue loss from outage. Goal is production-grade observability + scaling + cost controls.

System Design Overview

This implementation uses a closed loop of (1) metrics, (2) dashboards and alerts, (3) autoscaling, and (4) cost optimizations: **Prometheus** scrapes LLM service metrics at 15s intervals and collects custom LLM workload signals (latency histogram, request rate, queue depth, token counters, cache stats, GPU utilization, and cost). **Grafana** visualizes four dimensions of observability: performance, availability, usage, and cost, with SLA gauges and model-level breakdowns. **Alertmanager** (or Grafana alerting) sends notifications to Slack/PagerDuty for latency, error rate, availability, and cost anomalies. **Kubernetes HPA** scales LLM inference pods using custom metrics (queue depth and latency) with asymmetric scale up/down behavior to avoid flapping. **Cost Optimizer** reduces spend via response caching, intelligent model routing, and cost tracking metrics.

Requirement 1: Prometheus Monitoring Configuration

Deliverable: Prometheus configuration, custom metrics, service discovery, and alerting rules with thresholds.

Key Prometheus settings (excerpt)

```
# =====

global:
  scrape_interval: 15s           # How often to scrape targets
  evaluation_interval: 15s       # How often to evaluate rules
  scrape_timeout: 10s           # Timeout for scrape requests

  # Attach labels to all metrics
  external_labels:
    environment: 'production'
    service: 'globalai-llm'
    region: 'us-east-1'
# Alertmanager configuration
alerting:
  - job_name: 'llm-inference'

  # Service discovery via Kubernetes (if using K8s)
  # kubernetes_sd_configs:
  #   - role: pod
```

```

# namespaces:
#   names:
#     - llm-production
# Static targets for development
static_configs:
  - targets:
    - 'llm-server-1:8000'
    - 'llm-server-2:8000'
    - 'llm-server-3:8000'
metrics_path: /metrics
# Optional: filter to only GPU-related metrics
metric_relabel_configs:
  - source_labels: [__name__]
    regex: 'DCGM_.*'
    action: keep
# -----
# Node Exporter (System metrics)
- job_name: 'node-exporter'
  - 'node-exporter:9100'
  # Keep only essential system metrics
  regex: 'node_(cpu|memory|disk|network)_.*'
# Redis (for caching metrics)
- job_name: 'redis'
  - 'redis-exporter:9121'

```

Custom metrics expected from LLM services (exposed on /metrics):

Latency: llm_request_duration_seconds_bucket (histogram) for P50/P95/P99 via histogram_quantile.

Traffic: llm_request_total (counter) labeled by model, status, and endpoint. **Queue depth:**

llm_request_queue_depth (gauge) for proactive scaling (GPU boot time 4-5 min). **Tokens:**

llm_tokens_input_total and llm_tokens_output_total (counters). **GPU:** DCGM exporter metrics such as DCGM_FI_DEV_GPU_UTIL and memory usage for saturation alerts. **Cost:** llm_cost_dollars_total and llm_cost_per_request (derived) for anomaly detection. **Caching:** llm_cache_hits_total, llm_cache_misses_total, and hit ratio gauge.

Alerting rules (excerpt)

```

interval: 30s
rules:
  - alert: LLMHighLatencyWarning
    expr: |
      histogram_quantile(0.95,
        sum(rate(llm_request_duration_seconds_bucket{job="llm-inference"}[5m])) by (le, model)
      ) > 1.5
    for: 2m
    labels:
      severity: warning
      team: platform
    annotations:
      summary: "LLM latency approaching SLA threshold"
      description: "95th percentile latency for model {{ $labels.model }} is {{ $value | printf \"%.2f\" }}"
      runbook_url: "https://wiki.globalai.com/runbooks/llm-latency"

  - alert: LLMHighLatencyCritical
    expr: |
      histogram_quantile(0.99,
        sum(rate(llm_request_duration_seconds_bucket{job="llm-inference"}[5m])) by (le, model)
      ) > 2
    for: 1m
    labels:
      severity: critical
      pagerduty: "true"
    annotations:
      summary: "LLM latency SLA BREACHED"
      description: "99th percentile latency for model {{ $labels.model }} is {{ $value | printf \"%.2f\" }}"

  - alert: LLMHighErrorRate

```

```

(
  sum(rate(llm_request_total{status=~"5.."}[5m])) by (model)
  /
  sum(rate(llm_request_total[5m])) by (model)
) * 100 > 0.1
summary: "LLM error rate above SLA threshold"
runbook_url: "https://wiki.globalai.com/runbooks/llm-errors"

- alert: LLMRequestTimeouts
  sum(rate(llm_timeout_total[5m])) by (model) > 1
  summary: "LLM requests timing out"
  description: "Model {{ $labels.model }} has {{ $value | printf \"%.1f\\\" }} timeouts/sec"
  runbook_url: "https://wiki.globalai.com/runbooks/llm-timeouts"
# =====
- alert: LLMServiceDown
  expr: up{job="llm-inference"} == 0
  summary: "LLM inference server DOWN"
  description: "Instance {{ $labels.instance }} is not responding"
  runbook_url: "https://wiki.globalai.com/runbooks/llm-down"
- alert: LLMInsufficientReplicas
  count(up{job="llm-inference"} == 1) < 2
  summary: "Insufficient LLM replicas running"
  description: "Only {{ $value }} healthy replicas (minimum: 2)"
  runbook_url: "https://wiki.globalai.com/runbooks/llm-replicas"
- alert: GPUHighUtilization
  DCGM_FI_DEV_GPU_UTIL > 90
  for: 5m
  summary: "GPU utilization high"
  description: "GPU {{ $labels.gpu }} on {{ $labels.instance }} at {{ $value | printf \"%.0f\\\" }}%"
  runbook_url: "https://wiki.globalai.com/runbooks/gpu-utilization"
- alert: GPUMemoryPressure
  (DCGM_FI_DEV_FB_USED / DCGM_FI_DEV_FB_TOTAL) * 100 > 85
  summary: "GPU memory pressure"
  description: "GPU {{ $labels.gpu }} memory at {{ $value | printf \"%.0f\\\" }}%"
  runbook_url: "https://wiki.globalai.com/runbooks/gpu-memory"
- alert: HighRequestQueueDepth
  llm_request_queue_depth > 100
  summary: "LLM request queue building up"
  description: "Queue depth is {{ $value }} requests - consider scaling"
  runbook_url: "https://wiki.globalai.com/runbooks/queue-depth"
interval: 5m
- alert: DailyCostThresholdWarning
  sum(increase(llm_cost_dollars_total[24h])) > 8000
  team: finance
  summary: "Daily LLM costs approaching budget"

```

Requirement 2: Grafana Dashboard

Deliverable: Dashboard JSON export, four-dimension screenshots, alert notification configuration, and explanation of metric choices.

Dashboard coverage: The provided dashboard organizes SLA gauges and time-series panels across performance, availability, usage, cost, and infrastructure.

Section (Row)	Panel	Type
■ SLA Overview	24h Availability	gauge
■ SLA Overview	P99 Latency	gauge
■ SLA Overview	24h Cost	stat
■ SLA Overview	Current RPS	stat
■ SLA Overview	Queue Depth	stat
■ SLA Overview	Cache Hit Ratio	stat
■ Request Metrics	Request Rate by Model	timeseries
■ Request Metrics	Request Latency Percentiles	timeseries
■ Cost Metrics	Hourly Cost by Model	timeseries
■ Cost Metrics	Token Usage by Model	timeseries
■■ Infrastructure Metrics	GPU Utilization	timeseries
■■ Infrastructure Metrics	GPU Memory Usage	timeseries
■■ Infrastructure Metrics	LLM Server Replicas	timeseries

Key Grafana PromQL queries (excerpt)

```
# 24h Availability
(1 - (sum(rate(llm_request_total{status=~"5.."}[24h])) / sum(rate(llm_request_total[24h])))) * 100

# P99 Latency
histogram_quantile(0.99, sum(rate(llm_request_duration_seconds_bucket[5m])) by (le))

# 24h Cost
sum(increase(llm_cost_dollars_total[24h]))

# Current RPS
sum(rate(llm_request_total[5m]))

# Queue Depth
sum(llm_request_queue_depth)

# Request Rate by Model
sum(rate(llm_request_total[5m])) by (model)
```

Alert notifications: Configure Grafana alert contact points (Slack + PagerDuty) and route severity based on SLA breach vs warning thresholds.

Requirement 3: Auto-Scaling Configuration

Deliverable: HPA YAML, custom metrics adapter config, scaling timeline during load test, and latency graphs showing SLA compliance.

HPA configuration (excerpt)

```
# =====

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: llm-inference-hpa
  namespace: llm-production
  labels:
    app: llm-inference
    tier: production
spec:
  # Target deployment to scale
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: llm-inference

  # Replica limits
  minReplicas: 2          # Minimum for high availability
  maxReplicas: 10         # Maximum based on budget/capacity
  # Scaling metrics
  metrics:
    # -----
    # CPU-based scaling (baseline)
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
    # Scale UP behavior - React quickly to load increases
    scaleUp:
      stabilizationWindowSeconds: 60  # Wait 60s before scaling up again
      policies:
        - type: Percent
          value: 100                  # Can double replicas
          periodSeconds: 60
        - type: Pods
          value: 4                    # Or add up to 4 pods
          selectPolicy: Max           # Use whichever adds more pods

    # Scale DOWN behavior - Scale down slowly to avoid thrashing
    scaleDown:
      stabilizationWindowSeconds: 300  # Wait 5 minutes before scaling down
      value: 25                        # Remove max 25% of replicas
      periodSeconds: 120
      value: 1                         # Or remove 1 pod at a time
      selectPolicy: Min               # Use whichever removes fewer pods
  ---
  # Prometheus Adapter Configuration for Custom Metrics
```

Scaling triggers: queue depth and tail latency (P95) rather than CPU because GPU nodes have multi-minute provisioning delays. **Asymmetric policy:** scale up faster (add pods after ~2 minutes of breach) and scale down slower (remove pods only after sustained low load, e.g., 15 minutes). **Stabilization window:** 300 seconds to avoid oscillations during traffic volatility.

Requirement 4: Cost Optimization Implementation

Deliverable: Redis caching configuration and hit-rate metrics, model routing logic + distribution analysis, dynamic scaling schedule, and before/after cost comparison.

Cost optimizer module (excerpt)

```
# =====

CACHE_HITS = Counter(
    'llm_cache_hits_total',
    'Total number of cache hits',
    ['cache_type']
)
CACHE_MISSES = Counter(
    'llm_cache_misses_total',
    'Total number of cache misses',
)
COST_SAVED = Counter(
    'llm_cost_saved_dollars_total',
    'Total cost saved through optimization',
    ['optimization_type']
)
MODEL_REQUESTS = Counter(
    'llm_model_requests_total',
    'Requests by model',
    ['model', 'routing_reason']
)
"""Configuration for each model"""
name: str
tier: ModelTier
cost_per_1k_input_tokens: float
cost_per_1k_output_tokens: float
max_context_length: int
avg_latency_ms: float
# Model configurations
MODELS = {
    "llama-3.1-8b": ModelConfig(
        name="llama-3.1-8b",
        tier=ModelTier.FAST,
        cost_per_1k_input_tokens=0.0005,
        cost_per_1k_output_tokens=0.0015,
        max_context_length=8192,
        avg_latency_ms=200
    ),
    "llama-3.1-70b": ModelConfig(
        name="llama-3.1-70b",
        tier=ModelTier.STANDARD,
        cost_per_1k_input_tokens=0.003,
        cost_per_1k_output_tokens=0.015,
        avg_latency_ms=800
    ),
    "gpt-4-turbo": ModelConfig(
        name="gpt-4-turbo",
        tier=ModelTier.PREMIUM,
        cost_per_1k_input_tokens=0.01,
        cost_per_1k_output_tokens=0.03,
        max_context_length=128000,
        avg_latency_ms=1200
    )
}
# Semantic Cache Implementation
class SemanticCache:
    """
    Cache for LLM responses based on semantic similarity.
    Uses exact match by default; can be extended for embedding-based similarity.
    """
    def __init__(
```

```

        self,
        redis_url: str = "redis://localhost:6379",
        ttl_seconds: int = 3600, # 1 hour default
        max_cache_size_mb: int = 1000
    ):
        self.redis_client = redis.from_url(redis_url)
        self.ttl_seconds = ttl_seconds
        self.max_cache_size_mb = max_cache_size_mb
        self.cache_prefix = "llm_cache:"

        # Stats
        self.hits = 0
        self.misses = 0
    def _generate_cache_key(self, prompt: str, model: str) -> str:
        """Generate deterministic cache key from prompt and model"""
        content = f"{model}:{prompt.strip().lower()}"
        return f"{self.cache_prefix}{hashlib.sha256(content.encode()).hexdigest()}"

        self.redis_client.setex(
            cache_key,
            self.ttl_seconds,
            json.dumps(cache_data)
        )
        logger.info(f"Cached response for key {cache_key[:20]}...")
        return True
    except Exception as e:
        logger.error(f"Cache set error: {e}")
        return False
    def _update_hit_ratio(self):
# Intelligent Model Router
class ModelRouter:
    Routes requests to appropriate model based on complexity and cost.
    Uses heuristics to determine query complexity.
    def __init__(self, default_model: str = "llama-3.1-8b"):
        self.default_model = default_model
        # Simple queries that can use fast model
        self.simple_patterns = [
            "what is",
        ]
        return 0.0
    original_cost = (
        (input_tokens / 1000) * original_config.cost_per_1k_input_tokens +
        (output_tokens / 1000) * original_config.cost_per_1k_output_tokens
    )

```

Optimization strategy applied: Redis response caching with TTL: 24h for general Q&A; and 7 days for classification-style prompts (long-lived labels). Cache metrics track hits/misses and hit ratio. **Intelligent routing**: send simple prompts to a cheaper/fast model (Llama 3.1-8B) and complex prompts to a stronger model (Llama 3.1-70B). Routing decisions are logged and exported as metrics. **Dynamic schedule**: baseline capacity of 3 instances during business hours (08:00-18:00) and 1 instance off-hours, while keeping HPA active for spikes.

Traffic & cost signals from provided sample dataset (used to validate thresholds and dashboards):

Period	RPS	P99	Error	Cost	GPU util	Cache hit
Baseline (00:00Z)	45	450 ms	0.10%	\$12.5/min	35%	42%
Trough (low traffic)	12	340 ms	0.00%	\$3.8/min	10%	60%
Spike (10x+ event)	1500	18000 ms	18.00%	\$685.0/min	100%	4%

Cost anomaly baseline is defined as the median cost_per_minute during low traffic (RPS < 100): \$9.20/min. Alert threshold is baseline * 1.2 = \$11.04/min.

Requirement 5: Performance Testing Results

Deliverable: JMeter test plan, results showing 10x traffic handling, response time distribution, and 3-5 sentence business analysis.

JMeter test plan configuration (excerpt)

```
<!-- ===== -->
<!-- Baseline Load Test - Steady State -->
<ThreadGroup guiclass="ThreadGroupGui" testclass="ThreadGroup" testname="Baseline Load Test" enabled="true">
  <stringProp name="ThreadGroup.on_sample_error">continue</stringProp>
  <elementProp name="ThreadGroup.main_controller" elementType="LoopController">
    <boolProp name="LoopController.continue_forever">false</boolProp>
    <intProp name="LoopController.loops">-1</intProp>
  </elementProp>
  <stringProp name="ThreadGroup.num_threads">50</stringProp>
  <stringProp name="ThreadGroup.ramp_time">60</stringProp>
  <boolProp name="ThreadGroup.scheduler">true</boolProp>
  <stringProp name="ThreadGroup.duration">300</stringProp>
  <stringProp name="ThreadGroup.delay">0</stringProp>
  <boolProp name="ThreadGroup.same_user_on_next_iteration">true</boolProp>
</ThreadGroup>
<hashTree>

  <!-- HTTP Request Defaults -->
  <ConfigTestElement guiclass="HttpDefaultsGui" testclass="ConfigTestElement" testname="HTTP Request Defaults">
    <elementProp name="HTTPSampler.Arguments" elementType="Arguments">
      <collectionProp name="Arguments.arguments"/>
    </elementProp>
    <stringProp name="HTTPSampler.domain">${TARGET_HOST}</stringProp>
    <stringProp name="HTTPSampler.port">${TARGET_PORT}</stringProp>
    <stringProp name="HTTPSampler.protocol">http</stringProp>
    <stringProp name="HTTPSampler.contentEncoding">UTF-8</stringProp>
  </hashTree>

  <!-- LLM Inference Request -->
  <HTTPSamplerProxy guiclass="HttpTestSampleGui" testclass="HTTPSamplerProxy" testname="LLM Inference Request">
    <boolProp name="HTTPSampler.postBodyRaw">true</boolProp>
    <collectionProp name="Arguments.arguments">
      <elementProp name="" elementType="HTTPArgument">
        <boolProp name="HTTPArgument.always_encode">false</boolProp>
        <stringProp name="Argument.value">{
  "prompt": "${prompt}",
  "max_tokens": 512,
  "temperature": 0.7,
  "model_type": "customer_service"
}</stringProp>
        <stringProp name="HTTPSampler.embedded_url_re"></stringProp>
        <stringProp name="HTTPSampler.connect_timeout">5000</stringProp>
        <stringProp name="HTTPSampler.response_timeout">30000</stringProp>
      </elementProp>
    </collectionProp>
  </HTTPSamplerProxy>
  <hashTree>

    <!-- Response Assertion - Success -->
    <ResponseAssertion guiclass="AssertionGui" testclass="ResponseAssertion" testname="Response Code Assertion">
      <collectionProp name="Assertion.test_strings">
        <stringProp name="49586">200</stringProp>
      </collectionProp>
      <stringProp name="Assertion.custom_message">Expected HTTP 200 but got ${__groovy(prev.getResponseCode())}</stringProp>
    </ResponseAssertion>
  </hashTree>
</hashTree>
```



```

    <stringProp name="Assertion.test_field">Assertion.response_code</stringProp>
    <boolProp name="Assertion.assume_success">false</boolProp>
    <intProp name="Assertion.test_type">8</intProp>
<!-- Duration Assertion - SLA Check -->
<DurationAssertion guiclass="DurationAssertionGui" testclass="DurationAssertion" testname="SLA L
    <stringProp name="DurationAssertion.duration">${SLA_LATENCY_MS}</stringProp>
</DurationAssertion>
<hashTree/>
<!-- JSON Response Assertion -->
<JSONPathAssertion guiclass="JSONPathAssertionGui" testclass="JSONPathAssertion" testname="Respo
    <stringProp name="JSON_PATH">$.response</stringProp>
    <stringProp name="EXPECTED_VALUE"></stringProp>
    <boolProp name="JSONVALIDATION">false</boolProp>
    <boolProp name="EXPECT_NULL">false</boolProp>
    <boolProp name="INVERT">false</boolProp>
    <boolProp name="ISREGEX">false</boolProp>
</hashTree>
<!-- Health Check Request (10% of requests) -->
<HTTPSamplerProxy guiclass="HttpTestSampleGui" testclass="HTTPSamplerProxy" testname="Health Check
    <stringProp name="HTTPSampler.path">/health</stringProp>
    <stringProp name="HTTPSampler.method">GET</stringProp>
    <boolProp name="HTTPSampler.use_keepalive">true</boolProp>
    <ThroughputController guiclass="ThroughputControllerGui" testclass="ThroughputController" testna
        <intProp name="ThroughputController.style">1</intProp>
        <boolProp name="ThroughputController.perThread">false</boolProp>
        <intProp name="ThroughputController.maxThroughput">1</intProp>
    <FloatProperty>
        <name>ThroughputController.percentThroughput</name>
        <value>10.0</value>

```

How results are validated: Run the JMeter plan to ramp from 100 to 1000 concurrent users over 5 minutes (10x spike). In Kubernetes, capture HPA events (kubectl describe hpa and event timeline) and verify scale-up begins when queue depth and/or P95 breaches thresholds. In Grafana, export screenshots for: RPS, P50/P95/P99 latency, error rate, queue depth, replicas, GPU utilization, and cost-per-request. Confirm SLA: P95 latency stays below 1.5s target (or rapidly returns below target after brief warm-up), and error rate remains below 1%.

Results table (paste your observed numbers)

Metric	Target	Observed (fill in)	Notes
P95 latency	< 1.5s	_____	Tail latency primary SLO
Error rate	< 1%	_____	5xx / total
Queue depth	< 30	_____	Proactive scaling trigger
Replicas (min/max)	2 / 10	_____	Verify scale-up/down timing
Recovery time	< 10 min	_____	Time to return to SLO after spike
Cost reduction	40-60%	_____	Compare before vs after optimization

Business analysis (example, 3-5 sentences):

With queue-depth and tail-latency driven HPA, GlobalAI can absorb sudden 10x surges by scaling inference replicas before GPU saturation cascades into timeouts, improving availability toward the 99.95% uptime target.

Cost optimization via caching and model routing reduces expensive large-model calls for repeatable or simple requests, which is typically where a 40-60% reduction is achievable without degrading quality.

Going forward, I would continuously monitor P95/P99 latency, error rate, queue depth, cache hit ratio, model distribution, and cost-per-request, and tune alert thresholds using real baseline data to minimize false positives while catching true regressions early.

Appendix: Included Artifacts

M3_prometheus.yml (Prometheus scrape + service discovery) M3_alert_rules.yml (Latency, error, availability, cost anomaly alerts) M3_grafana_dashboard.json (Four-dimensional dashboard export) M3_hpa.yml (HPA + scaling behavior) M3_load_test.jmx (JMeter 10x spike test plan) M3_cost_optimizer.py (Caching + routing + cost metrics) M3_traffic_patterns.csv, M3_test_prompts.csv (Data used to reason about thresholds and routing)

Note: This report documents the configuration and validation approach using the provided starter artifacts. Replace the 'Observed' column in the Results table with your measured values from your environment.